

Efficient Dependency Resolution in IFC-aware Decentralized Programming

Steffan Christ Sølvesten and Aslan Askarov

PRiSC 2026



Module systems are difficult!

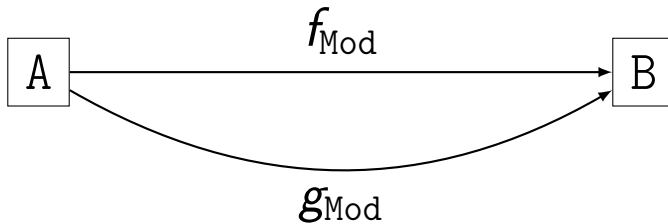
A

B

¹Using hashes to uniquely identify an object/class is called a “*fingerprint*” in O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).



¹Using hashes to uniquely identify an object/class is called a “*fingerprint*” in O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).



¹Using hashes to uniquely identify an object/class is called a “*fingerprint*” in O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

How can one add a module cache?

How can one add a *dependency* ~~module~~ cache?

Leaky Cache² (Confidentiality)

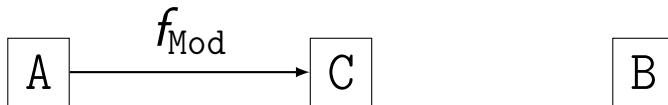
A

C

B

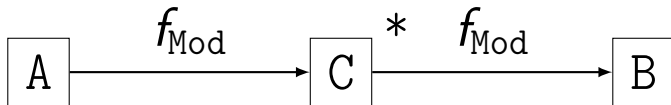
²This behaviour is a remanifestation of a “*read channel*” in
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

Leaky Cache² (Confidentiality)



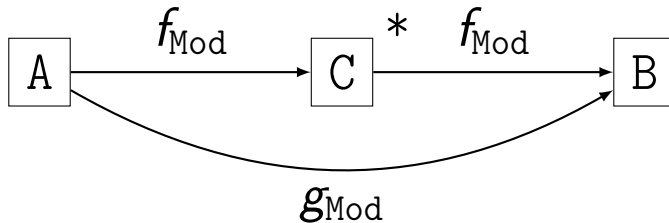
²This behaviour is a remanifestation of a “*read channel*” in
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

Leaky Cache² (Confidentiality)



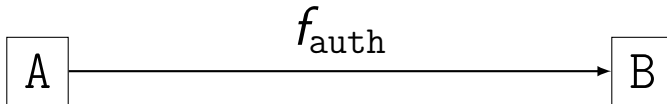
²This behaviour is a remanifestation of a “*read channel*” in
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

Leaky Cache² (Confidentiality)



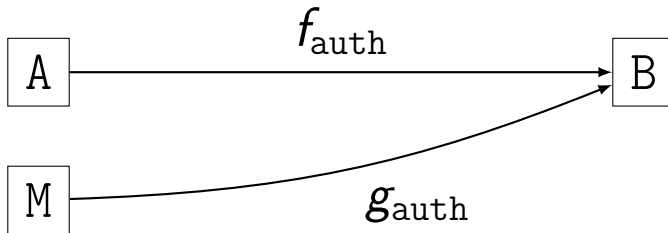
²This behaviour is a remanifestation of a “read channel” in
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

Replay Dependency³ (Integrity)



³This behavior is a remanifestation of the “*provider-bounded authority*” requirement in O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

Replay Dependency³ (Integrity)



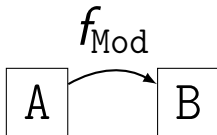
³This behavior is a remanifestation of the “*provider-bounded authority*” requirement in O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

Chain of Death⁴ (Availability)



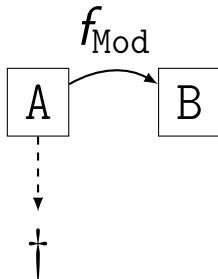
⁴This is related to the notion of “*mobile code*”. But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death⁴ (Availability)



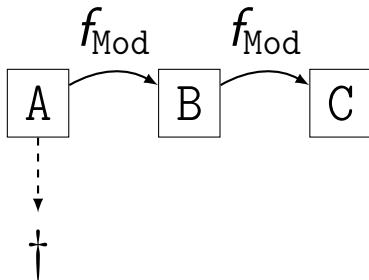
⁴This is related to the notion of “*mobile code*”. But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death⁴ (Availability)



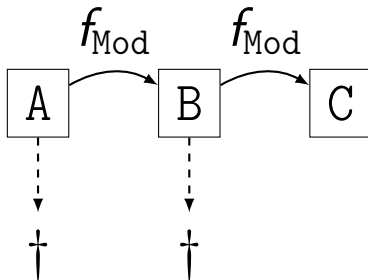
⁴This is related to the notion of “*mobile code*”. But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death⁴ (Availability)



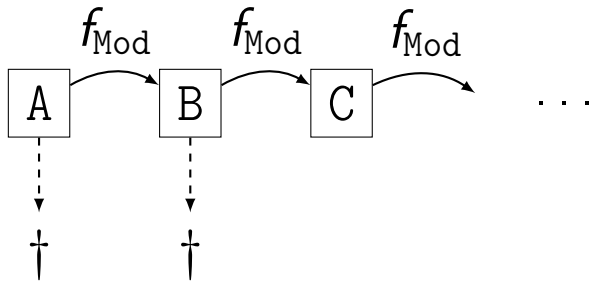
⁴This is related to the notion of “*mobile code*”. But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death⁴ (Availability)



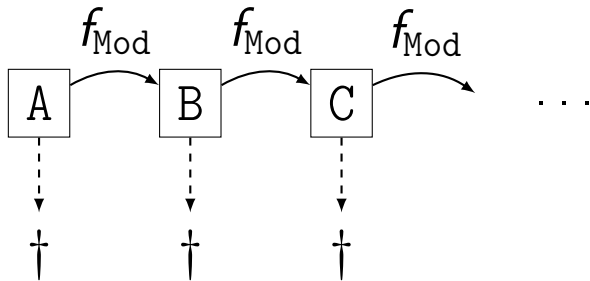
⁴This is related to the notion of “*mobile code*”. But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death⁴ (Availability)



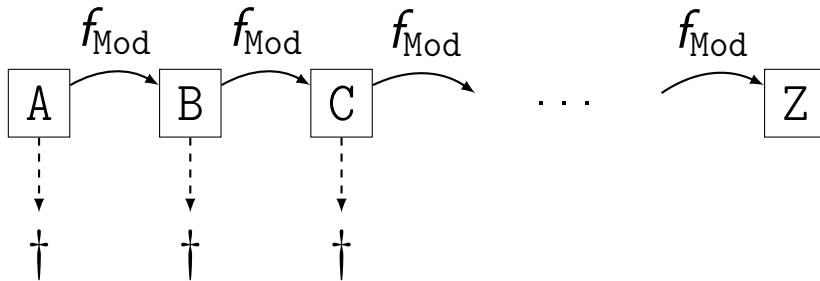
⁴This is related to the notion of “*mobile code*”. But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death⁴ (Availability)



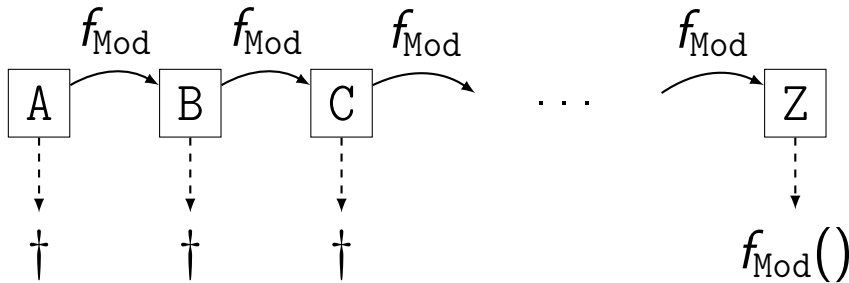
⁴This is related to the notion of “*mobile code*”. But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death⁴ (Availability)



⁴This is related to the notion of “*mobile code*”. But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death⁴ (Availability)



⁴This is related to the notion of “*mobile code*”. But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

How can one add an IFC-aware cache?

Troupe Programming Language

```
(* Alice.trp *)
```

```
1 let val msg = "Hello, Bob!"
2     fun f () = print msg
3 in send (@bob, f)
4 end
```

```
(* Bob.trp *)
```

```
5 let fun loop () =
6     receive [hn f => spawn f];
7     loop ()
8 in loop ()
9 end
```

■ Decentralized Programming

For creation of open self-organizing distributed systems.

■ Information Flow Control

Ensures via a monitor that secret information is not by accident shared with nor affected by unwanted parties.

■ Dynamically Typed

Permits the dynamic behaviour needed by a truly open and updateable system.

How can one add an IFC-aware cache?

How can one add an IFC-aware cache?
(with the minimal extension to the TCB...)

How can one add an IFC-aware cache?
(by using Troupe itself...?)

Alice

cache.trp

TCB

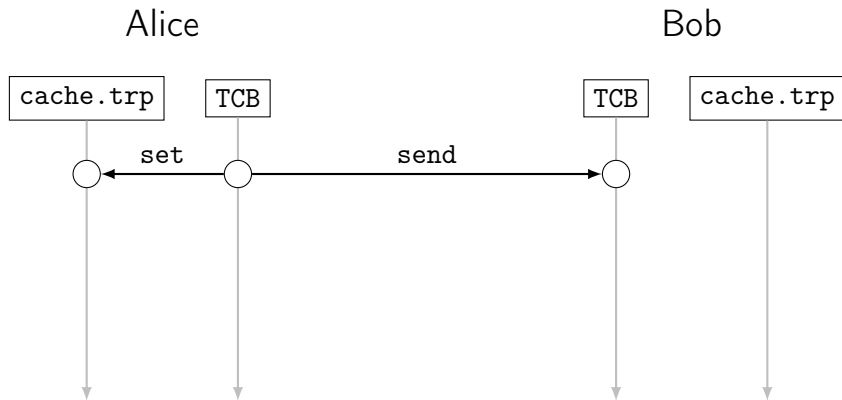


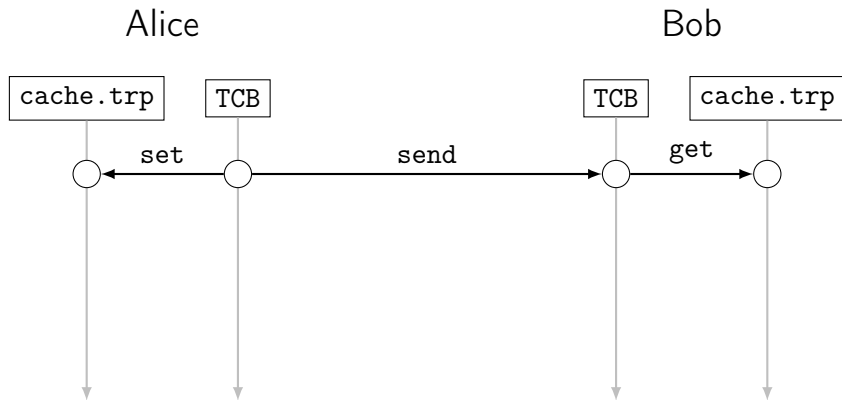
Bob

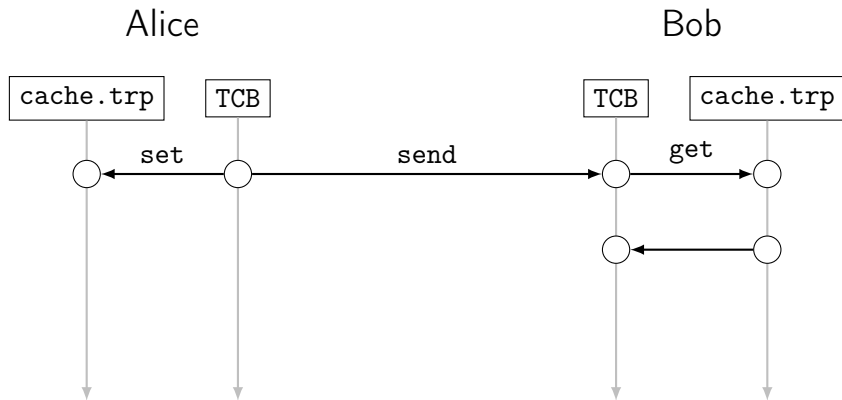
TCB

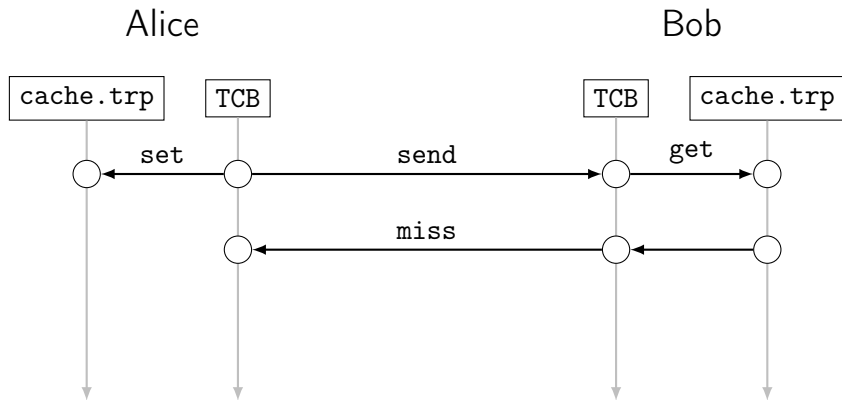
cache.trp

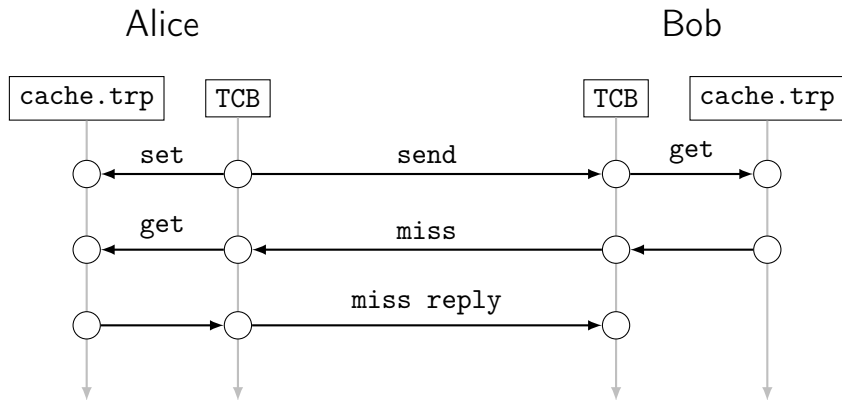


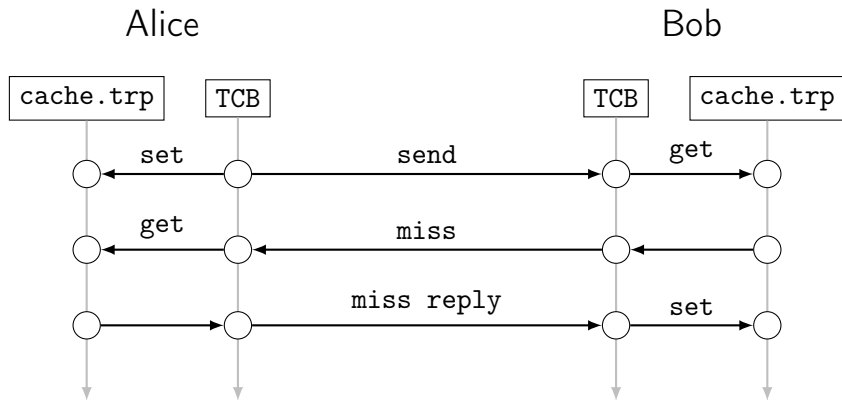












Additions in Troupe: caching layer

`cache.trp` is only a tiny Troupe program that orchestrate a `HashMap` instance.

Additions in Troupe: caching layer

`cache.trp` is only a tiny Troupe program that orchestrate a `HashMap` instance.

- Hash function for `HashMap`: `SHA256 string` $\rightarrow$ 32-bit int. (4 LOC)

Additions in Troupe: caching layer

`cache.trp` is only a tiny Troupe program that orchestrate a `HashMap` instance.

- Hash function for `HashMap`: `SHA256 string` $\rightarrow$ `32-bit int`. (4 LOC)
- Define and spawn a local Troupe thread that owns the `HashMap`. (21 LOC)

```
(* cache.trp : line 48 *)
```

```
HashMap.insert m auth k v
```

```
(* cache.trp : line 57 *)
```

```
HashMap.findAll m auth k
```

Additions in Troupe: caching layer

`cache.trp` is only a tiny Troupe program that orchestrate a `HashMap` instance.

- Hash function for `HashMap`: `SHA256 string` $\rightarrow$ `32-bit int`. (4 LOC)
- Define and spawn a local Troupe thread that owns the `HashMap`. (21 LOC)

```
(* cache.trp : line 48 *)      (* cache.trp : line 57 *)  
HashMap.insert m auth k v      HashMap.findAll m auth k
```

- Define and export `set` and `get` functions callable from the TCB. (8 LOC)

```
(** API Setter function *)  
fun set (key, value) = send(thread, (SET, key, value))
```

Additions in Troupe: caching layer

`cache.trp` is only a tiny Troupe program that orchestrate a `HashMap` instance.

- Hash function for `HashMap`: `SHA256 string` $\rightarrow$ `32-bit int`. (4 LOC)
- Define and spawn a local Troupe thread that owns the `HashMap`. (21 LOC)

```
(* cache.trp : line 48 *)      (* cache.trp : line 57 *)  
HashMap.insert m auth k v      HashMap.findAll m auth k
```

- Define and export `set` and `get` functions callable from the TCB. (8 LOC)

```
(** API Setter function *)  
fun set (key, value) = send(thread, (SET, key, value))
```

- Use `sandbox builtin` to guard against timing attacks. (14 LOC)

Additions in Troupe: IFC-aware Map / HashMap

$$C \xrightarrow{f_{\text{Mod}}@ \{C\}} B \implies \text{miss} \quad A \xrightarrow{g_{\text{Mod}}@ \{\}} B \implies \text{miss} \quad D \xrightarrow{h_{\text{Mod}}@ \{D\}} B$$

Additions in Troupe: IFC-aware Map / HashMap

$$C \xrightarrow{f_{\text{Mod}}@ \{C\}} B \implies \text{miss} \quad A \xrightarrow{g_{\text{Mod}}@ \{\}} B \implies \text{miss} \quad D \xrightarrow{h_{\text{Mod}}@ \{D\}} B \implies \text{hit}$$

Additions in Troupe: IFC-aware Map / HashMap

$$C \xrightarrow{f_{\text{Mod}}@ \{C\}} B \implies \text{miss} \quad A \xrightarrow{g_{\text{Mod}}@ \{\}} B \implies \text{miss} \quad D \xrightarrow{h_{\text{Mod}}@ \{D\}} B \implies \text{hit}$$

“Specification”:

An IFC-aware (Hash)Map is a single data structure with public and secret key-value pairs which provides a IFC-aware projections of its insert and remove⁵ history:

$$i(x \mapsto 1)@ \{\}, \quad i(x \mapsto 2)@ \{A\}, \quad i(y \mapsto 3)@ \{B\}, \quad r(x)@ \{B\}, \quad i(y \mapsto 4)@ \{\}, \quad \dots$$

⁵For this particular use case, we actually only need public (@{\}) remove operations.

Additions in Troupe: IFC-aware Map / HashMap

$$C \xrightarrow{f_{\text{Mod}}@{\{C\}}} B \implies \text{miss} \quad A \xrightarrow{g_{\text{Mod}}@{\{\}}} B \implies \text{miss} \quad D \xrightarrow{h_{\text{Mod}}@{\{D\}}} B \implies \text{hit}$$

“Specification”:

An IFC-aware (Hash)Map is a single data structure with public and secret key-value pairs which provides a IFC-aware projections of its insert and remove⁵ history:

$$\begin{array}{c} i(x \mapsto 1)@{\{\}}, \quad i(x \mapsto 2)@{\{A\}}, \quad i(y \mapsto 3)@{\{B\}}, \quad r(x)@{\{B\}}, \quad i(y \mapsto 4)@{\{\}}, \quad \dots \\ \parallel \\ @{\{\}} \\ \Downarrow \\ [x \mapsto 1@{\{\}}, y \mapsto 4@{\{\}}] @{\{\}} \end{array}$$

⁵For this particular use case, we actually only need public (@{\}) remove operations.

Additions in Troupe: IFC-aware Map / HashMap

$$C \xrightarrow{f_{\text{Mod}}@{\{C\}}} B \implies \text{miss} \quad A \xrightarrow{g_{\text{Mod}}@{\{\}}} B \implies \text{miss} \quad D \xrightarrow{h_{\text{Mod}}@{\{D\}}} B \implies \text{hit}$$

“Specification”:

An IFC-aware (Hash)Map is a single data structure with public and secret key-value pairs which provides a IFC-aware projections of its insert and remove⁵ history:

$$\begin{array}{c} i(x \mapsto 1)@{\{\}}, \quad i(x \mapsto 2)@{\{A\}}, \quad i(y \mapsto 3)@{\{B\}}, \quad r(x)@{\{B\}}, \quad i(y \mapsto 4)@{\{\}}, \quad \dots \\ \parallel \\ @{\{A\}} \\ \Downarrow \\ [x \mapsto 2@{\{A\}}, y \mapsto 4@{\{\}}] @{\{A\}} \end{array}$$

⁵For this particular use case, we actually only need public (@{\}) remove operations.

Additions in Troupe: IFC-aware Map / HashMap

$$C \xrightarrow{f_{\text{Mod}}@ \{C\}} B \implies \text{miss} \quad A \xrightarrow{g_{\text{Mod}}@ \{\}} B \implies \text{miss} \quad D \xrightarrow{h_{\text{Mod}}@ \{D\}} B \implies \text{hit}$$

“Specification”:

An IFC-aware (Hash)Map is a single data structure with public and secret key-value pairs which provides a IFC-aware projections of its insert and remove⁵ history:

$$\begin{array}{c} i(x \mapsto 1)@ \{\}, \quad i(x \mapsto 2)@ \{A\}, \quad i(y \mapsto 3)@ \{B\}, \quad r(x)@ \{B\}, \quad i(y \mapsto 4)@ \{\}, \quad \dots \\ \parallel \\ @ \{B\} \\ \Downarrow \\ [y \mapsto 4@ \{\}] \quad @ \{B\} \end{array}$$

⁵For this particular use case, we actually only need public (@{\}) remove operations.

Additions in Troupe: IFC-aware Map / HashMap

$$C \xrightarrow{f_{\text{Mod}}@ \{C\}} B \implies \text{miss} \quad A \xrightarrow{g_{\text{Mod}}@ \{\}} B \implies \text{miss} \quad D \xrightarrow{h_{\text{Mod}}@ \{D\}} B \implies \text{hit}$$

“Specification”:

An IFC-aware (Hash)Map is a single data structure with public and secret key-value pairs which provides a IFC-aware projections of its insert and remove⁵ history:

$$\begin{array}{c} i(x \mapsto 1)@ \{\}, \quad i(x \mapsto 2)@ \{A\}, \quad i(y \mapsto 3)@ \{B\}, \quad r(x)@ \{B\}, \quad i(y \mapsto 4)@ \{\}, \quad \dots \\ \parallel \\ @ \{A, B\} \\ \Downarrow \\ [y \mapsto 4@ \{\}] \quad @ \{A, B\} \end{array}$$

⁵For this particular use case, we actually only need public (@{\}) remove operations.

Changes to Runtime (TCB)

- **Peer-to-peer layer:**

Extend with `miss` and `miss reply` messages which are always sent by value.

- **Serialization:**

Dependency resolution requires introspection of function closures.

- During `serialize`, identify (sub)values to be referenced by hash.
- To `deserialize`, (1) extract references (2) `deserialize` given (all resolved) dependencies.

- **Hash:**

Hashing Troupe values requires introspection of function closures and source.

- **Scheduler:**

Extend to run Troupe code from the peer-to-peer layer.

How can one add an IFC-aware cache?

How can one add an IFC-aware cache?

By bootstrapping the IFC-aware runtime!

Steffan Christ Sølvsten

✉ soelvsten@cs.au.dk

🌐 ssoelvsten.github.io

Troupe

🔗 github.com/TroupeLang/troupe



Troupe's Threat Model

Uses several security mechanisms, inspired by Jif, Fabric, and FLAM.

- Progress-sensitive Dynamic Information-flow Control
- Sandboxing
- Capabilities

Few assumptions on adversaries! But, we do ignore:

- Network Traffic Analysis. Can be avoided with via recent methods.
(Blaabjerg and Askarov '21 & '23, Nelson, Pagnin, and Askarov '24)
- Timing-channels. Can be avoided via sandboxing and predictive mitigation.
(Zhang, Askarov, Myers '10 & '12)

Additions in Troupe: caching layer (timing-attack mitigation)

Troupe provides the `sandbox(f, t)` operation to run *f* for *t* ms and either return the result, an error, or a continuation. In practice, *t* = 1 ms seems to suffice.

```
(* cache.trp : line 48 *)  
val (m', t') =  
    sandbox' (fn () => HashMap.insert m auth k v) t m  
  
(* cache.trp : line 57 *)  
val (v, t') =  
    sandbox' (fn () => HashMap.findAll m auth k) t []
```

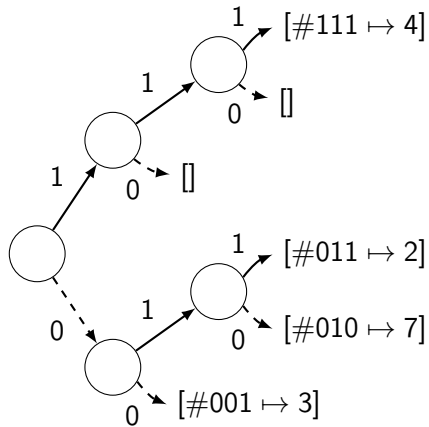

Design of the HashMap

Current prototype uses a Hash Array
Mapped Trie (HAMT) with 200 LOC.

Only 3 declassifications per insert,
remove, find (-24/+55 LOC):

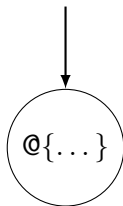
- hash key and hash bucket to not raise pc and taint the entire trie.
- The bucket list (but not its content).

But, this fully declassifies its size!



Design of the HashMap

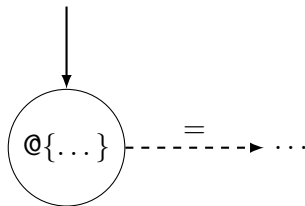
Can we design a meta data structure for
HashMap and Map?



Design of the HashMap

Can we design a meta data structure for HashMap and Map?

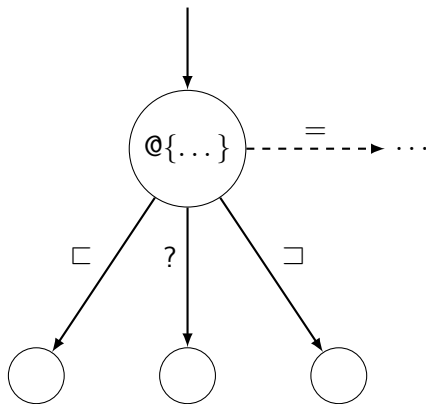
- Guard taint of secret sub structures with (first-class) IFC labels.



Design of the HashMap

Can we design a meta data structure for HashMap and Map?

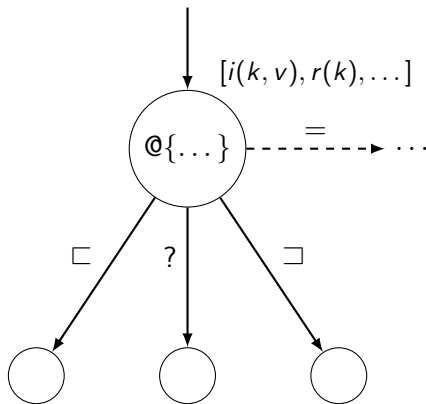
- Guard taint of secret sub structures with (first-class) IFC labels.
- Use the partial ordering on information flow ($\sqsubseteq$) to compare levels.



Design of the HashMap

Can we design a meta data structure for HashMap and Map?

- Guard taint of secret sub structures with (first-class) IFC labels.
- Use the partial ordering on information flow ($\sqsubseteq$) to compare levels.
- Use buffers to accumulate operations for “inaccessible” sub structures. This keeps `insert` and `remove` efficient.



Design of the HashMap

Can we design a meta data structure for HashMap and Map?

- Guard taint of secret sub structures with (first-class) IFC labels.
- Use the partial ordering on information flow ($\sqsubseteq$) to compare levels.
- Use buffers to accumulate operations for “inaccessible” sub structures. This keeps `insert` and `remove` efficient.
- Use time-stamps to identify latest key-value from sub structures.

But, does this make `find` too slow?

